

Q1. Given an array [N], every element repeats twice except 1.
Find the unique element.

ex $\Rightarrow$ arr[] $\Rightarrow$ { 3, 6, 4, 4, 5, 3, 5 }

o/p $\Rightarrow$ 6.

$$\underline{a \wedge b \wedge c \wedge d} \Rightarrow \underline{d \wedge c \wedge a \wedge b}$$

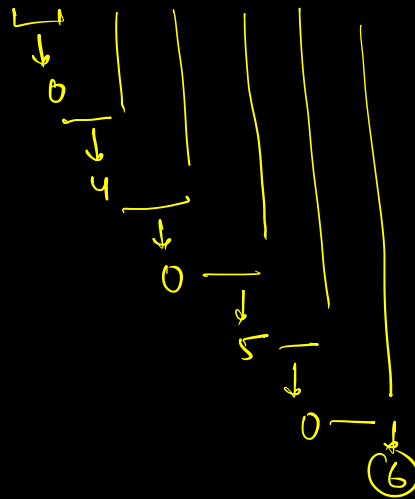
$$a \wedge 0 \Rightarrow a$$

$$\begin{array}{c} a \wedge a \\ \text{---} \end{array} \Rightarrow \underline{0}$$

{ 3, 6, 4, 4, 5, 3, 5 }

$\Rightarrow$ { 3, 3, 4, 4, 5, 5, 6 }

O/p = 6.



TC $\Rightarrow O(N)$
SC $\Rightarrow O(1)$

Q2. Given arr[N], there are two unique element, every other element is repeating twice, find the unique elements

ex $\Rightarrow$ arr[6] $\Rightarrow$ { 3, 6, 4, 4, 3, 8 } $\Rightarrow$ o/p = 6, 8

arr[4] $\Rightarrow$ { 4, 9, 9, 8 } $\Rightarrow$ o/p = 4, 8.

Solⁿ 1

Create a freq. hashmap, return the arr^e elements having freq = 1

TC $\Rightarrow O(N)$
SC $\Rightarrow O(N)$

Solⁿ 2

Sort the array, and check for unique nos.

TC $\Rightarrow O(N \log N)$

SC $\Rightarrow$ $\rightarrow$ depends on sorting algo.

Idea 3

XOR of all elements

{ 3, 6, 4, 4, 3, 8 } $\Rightarrow 6 \wedge 8 \Rightarrow 14$

arr[] $\Rightarrow$ 10 8 8 9 12 9 6 11 10 6 12 17

(1010) (1000) (1100) (0110) (1010) (1100)
 (1000) (1001) (1001) (1011) (0110) (10001)

XOR of

all elements $\Rightarrow 11 \wedge 17$

11: 0 1 0 1 1

17: 1 0 0 0 1

11 $\wedge$ 17 $\Rightarrow$ 1 1 0 1 0

obs at bit pos: 1, 3, 4

both unique elements

11 217 have different

bits.

Bit pos = 1 [reference]

set

unset

$\left\{ \begin{array}{l} 10 \ 6 \ 11 \ 10 \\ 6 \end{array} \right\}$
 $\left\{ \begin{array}{l} 8 \ 8 \ 9 \ 12 \\ 9 \ 12 \ 17 \end{array} \right\}$

$\downarrow$
 XOR of
 all
 $\downarrow$
 11
 $\checkmark$

$\downarrow$
 XOR of
 all
 $\downarrow$
 17
 $\checkmark$

Bit pos (3) $\Rightarrow$ reference

set

unset

$\left\{ \begin{array}{l} 10, 8, 8, 9, 12, \\ 9, 11, 10, 12 \end{array} \right\}$
 $\left\{ \begin{array}{l} 6, 6, 17 \end{array} \right\}$

$\downarrow$
 XOR
 $\downarrow$
 11

$\downarrow$
 XOR
 $\downarrow$
 17

pseudo code

1) xor of all elements

val = 0

→ $O(N)$

```
for (i = 0; i < N; i++) {  
    val = val ^ arr[i]  
}
```

11) find the set bit in val
pos.

pos = -1

```
for (i = 0; i < 31; i++) {  
    if (checkbit(i, val)) {  
        pos = i  
        break  
    }  
}
```

$O(1)$

→

111) using bit pos 'pos' break the
array into set & unset
and xor for all

set = 0, unset = 0.

```
for (i = 0; i < N; i++) {  
    if (checkbit(pos, arr[i]))  
        set = set ^ arr[i]  
    else  
        unset = unset ^ arr[i]  
}
```

$O(N)$

→

111) return the result

{ unset, set }

$TC \Rightarrow O(2N + 31)$
 $\approx O(N)$
 $SC \Rightarrow O(1)$

Q3. Given $arr[N]$, contains all elements from 1 to $N+2$, except two missing elements, find the missing elements.

ex $\Rightarrow$ $arr[4] \Rightarrow \{3, 6, 1, 4\} \Rightarrow O/p \Rightarrow \{2, 5\}$

$\downarrow$
[1 to 6]

$arr[5] \Rightarrow \{1, 6, 4, 7, 5\} \Rightarrow O/p \Rightarrow \{2, 3\}$

$\downarrow$
1 to 7

$arr[8] \Rightarrow \{1, 3, 8, 5, 6, 4, 9, 10\} \Rightarrow \{2, 7\}$

$\downarrow$
1 to 10

Soln 1 $\Rightarrow$ Sort and find missing elements

TC $\Rightarrow O(N \log N)$

SC $\Rightarrow$ depends

Soln 2 $\Rightarrow$ using extra space

Hashmap [1 to $N+2$]

boolean array

TC $\Rightarrow O(N)$
SC $\Rightarrow O(N)$

Step 3

$$\begin{array}{l} 1 \text{ to } 7 \Rightarrow \{ \underline{1}, \underline{2}, \underline{3}, \underline{4}, \underline{5}, \underline{6}, \underline{7} \} \\ \text{arr}[5] \Rightarrow \{ \underline{1}, \underline{6}, \underline{4}, \underline{7}, \underline{5} \} \end{array}$$

Step 1 $\Rightarrow$ XOR of all elements

$$\begin{array}{l} \text{val} \swarrow \left[\begin{array}{l} \text{XOR} [1 \text{ to } N+2] \Rightarrow \underline{x} \\ \text{XOR} [\text{all elements of arr with } x] \end{array} \right. \end{array}$$

Step 2 $\Rightarrow$ same as prev. problem

Step 3 $\Rightarrow$ set and unset $\rightarrow$

1) loop 1 to $N+2$

2) loop the entire array

Step 4 $\Rightarrow$ return $\{ \text{set}, \text{unset} \}$

How

Q4, Given N elements, calculate sum of sum of all pairs.

arr[5] = { 3, 5, 6, 8, 2 }

brute force

sum = 0

for (i = 0; i < N; i++) {

for (j = 0; j < N; j++) {

sum = sum + (arr[i] * arr[j])

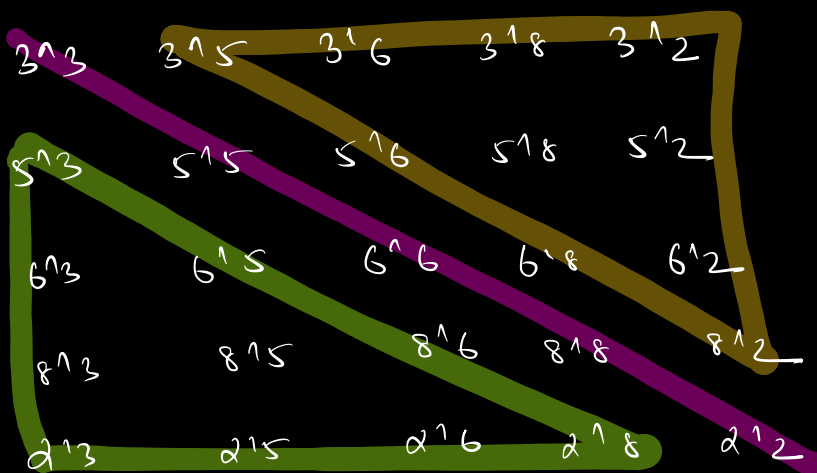
}

return sum;

TC $\Rightarrow O(N^2)$

SC $\Rightarrow O(1)$

{ 3, 5, 6, 8, 2 }



optimisation

compute sum of upper Δ [sum]

ans = 2 * sum

TC $\Rightarrow O(N^2)$

SC $\Rightarrow O(1)$

	3	2	1	0	
	2^3	2^2	2^1	2^0	
3^5	0	1	1	0	$\rightarrow 0 + 2^2 + 2^1 + 0$
3^6	0	1	0	1	$\rightarrow 0 + 2^2 + 0 + 2^0$
3^8	1	0	1	1	$\rightarrow 2^3 + 0 + 2^1 + 2^0$
3^2	0	0	0	1	$\rightarrow 0 + 0 + 0 + 2^0$
5^6	0	0	1	1	$\rightarrow 0 + 0 + 2^1 + 2^0$
5^8	1	1	0	1	$\rightarrow 2^3 + 2^2 + 0 + 2^0$
5^2	0	1	1	1	$\rightarrow 0 + 2^2 + 2^1 + 2^0$
6^8	1	1	1	0	$\rightarrow 2^3 + 2^2 + 2^1 + 0$
6^2	0	1	0	0	$\rightarrow 0 + 2^2 + 0 + 0$
8^2	1	0	1	0	$\rightarrow 2^3 + 0 + 2^1 + 0$

4	6	6	6
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
			2^0
			2^1
	2^2		
2^3			

$4 + 2^3 + 6 \times 2^2 + 6 \times 2^1 + 6 \times 2^0$
$\downarrow$
sum
$Ans = 2 \times \underline{\text{sum}}$

obs if for i^{th} bit pos, C_i is the number of xor pairs having i^{th} bit as set —

contribution $\Rightarrow [C_i \times 2^i]$

{ 3, 5, 6, 8, 2 }

3: 0011 5: 0101 6: 0110 8: 1000 2: 0010
 3 2 1 0 3 2 1 0 3 2 1 0 3 2 1 0

	set	unset	pairs
for bit 0	2	3	6
for bit 1	3	2	6
for bit 2	2	3	6
for bit 3	1	4	4

$$\begin{aligned} \text{sum} &= 6 \times 2^0 \\ &+ 6 \times 2^1 \\ &+ 6 \times 2^2 \\ &+ 4 \times 2^3 \end{aligned}$$

$$\text{Ans} = \underline{\underline{2 \times \text{sum}}}$$

pseudo

```

sum = 0
for (i = 0; i < 32; i++) {
    c = 0 // no. of elements having bit pos i, as set
    for (j = 0; j < N; j++) {
        if (checkBit(i, arr[j])) {
            c++
        }
    }
}

```

$$\text{sum} = \text{sum} + [C * (N-C) * (1 \leq i)]$$

} return sum2

TC $\Rightarrow O(N)$
SC $\Rightarrow O(1)$